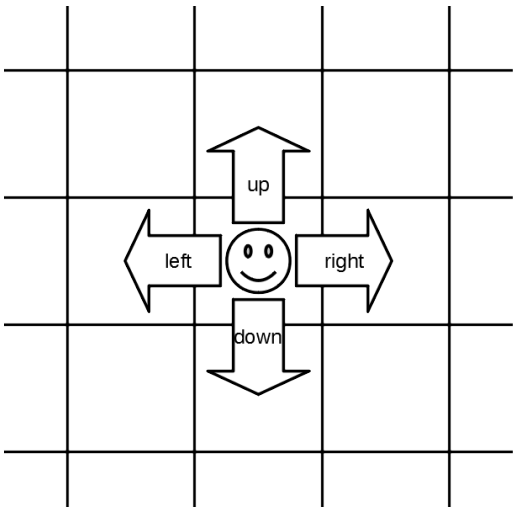
Aalborg University
Daniel Fišer

Exercise Sheet 2

1 Heuristics

1.1. Consider an unbounded regular 2D grid. The start state is at the origin, $(0, 0)$, and the goal state is at (x, y). You can move left, right, up and down. For example, if you are at the start and move to the right, you get to position $(1, 0)$. Moving down gets you to position $(0, -1)$. Each move has unit cost.



1. Is $h((u, v)) = |u - x| + |v - y|$ a consistent heuristic for a state at (u, v)? Justify your answer (1-2 sentences).
2. Is $h((u, v)) = |u - x| + |v - y|$ an admissible heuristic for a state at (u, v)? Justify your answer (1-2 sentences).
3. At most how many nodes are expanded by A* graph search using the Manhattan distance h_{Man} as your heuristic function? *You may assume that the Manhattan distance is admissible for this state space.*
4. Does h_{Man} remain admissible if some links are removed?
5. Does h_{Man} remain admissible if some links are added between nonadjacent states?

Solution:

1. Yes, this is a consistent heuristic; it is the Manhattan distance metric so for all transitions $s \xrightarrow{a} s'$ we have $h(s) - h(s') \leq c(a)$.

2. Yes, this is an admissible heuristic; it is the Manhattan distance metric (*alternatively*: consistency implies admissibility which was shown in 1.).

3. All nodes in the rectangle spanned by the corner nodes $(0, 0)$ and (x, y) are candidates for the optimal path and could be explored by A* graph search. Three are exactly $(|x| + 1) \cdot (|y| + 1)$ states in this rectangle and in the worst case, A* may explore every single one of them.

4. Yes; removing links may induce detours, which require more steps. So the heuristic h is an underestimate and hence still admissible.

5. No; nonlocal links can reduce the actual path length below the Manhattan distance. For example, if $(1, 1)$ was the goal state and an extra link between $(0, 0)$ and $(1, 1)$ has been added: then $h((0, 0)) = 2$, but the actual shortest path has length 1.

1.2. Consider the **Meeting Point** problem from Exercise Sheet 1, Exercise 1.3.

Let us model this problem as follows. States are encoded with variables a, e and f describing locations of Alice, Emil and Frank, respectively. We can assign each variable the location $A, B, \ldots, H$. The initial state is then $(a = A, e = E, f = F)$. Each action moves exactly one friend to a connected location with the cost according to the graph of locations.

Design a heuristic function for this problem, and argue why it is or is not admissible and/or consistent.

Solution: There are many options. To give one:

The cost of the cheapest path between two locations occupied by at least one friend.

It is admissible because they need to meet somewhere, and so they need to move (together) at least that amount (although it can still underestimate a lot).

Is is not consistent: Consider state $s = (a = D, e = D, f = C)$ and one of its successors $s' = (a = A, e = D, f = C)$. We have that $h(s) = 4$, $h(s') = 1$ and cost of the transition from s to s' is 1. Therefore we have that $h(s) - h(s') = 3$, but for a consistent h we would need to have $h(s) - h(s') \leq 1$.

The cost of the cheapest path between two friends would give us both admissible and consistent heuristic, but it would also be 0 whenever two friends meet at the same location.

1.3. Consider the **Elevator Scheduling** problem from Exercise Sheet 1, Exercise 1.4.

Let us encode states with variables $e, p_1, \ldots, p_P$ corresponding to the elevator and passengers 1 to P, respectively. The variable e have values $1, \ldots, F$ indicating the floor where is the elevator, and variables p_i have values $1, \ldots, F, e$ where $1, \ldots, F$ indicate on which floor the passenger waits for the elevator, and e indicates that the passenger is on-board the elevator. For example, the initial state for the instantiation from Exercise Sheet 1, would thus be encoded as $(e = 1, p_1 = 1, p_2 = 1, p_3 = 3)$, and if we board p_1 to the elevator, we get the state $(e = 1, p_1 = e, p_2 = 1, p_3 = 3)$.

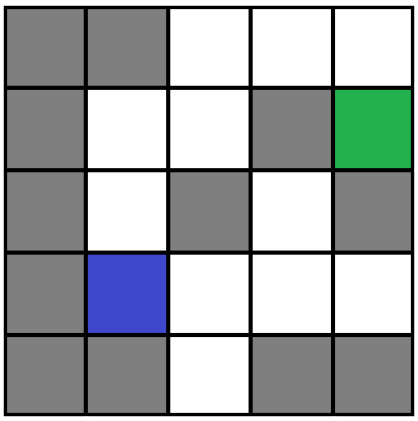
Actions are for boarding and leaving the elevator by one passenger (if they are both at the same floor) with cost 0, and moving of the elevator by one floor with cost 1.

Design an admissible and consistent heuristic that will work for any instantiation of the problem.

Solution: Let d_i denote the destination floor of the passenger i. Let D be the function defined as $D(i) = |p_i - d_i|$ if $p_i \in \{1, 2, 3\}$, and $D(i) = |e - d_i|$ if $p_i = e$. Denoting $h(e, p_1, \ldots, p_P)$ the h-value for the state $(e, p_1, \ldots, p_P)$, we define $h(e, p_1, \ldots, p_P) = \max_{i \in \{1, \ldots, P\}} D(i)$. In other words, we compute for each passenger the distance they need to travel by the elevator and then take the maximum over all passengers.

2 Systematic Search

2.1. Consider the labyrinth depicted below, in which the agent starts at the blue square and tries to get to the green one. The agent can only move up, down, left and right (**not** diagonally). The agent can walk on the white, green and blue squares, but not on the grey ones. Each square can be written down as a combination of its X and Y coordinates as (X, Y), which begin at the bottom left at 0. For example, the bottom left ist $(0, 0)$ and the goal is $(4, 3)$.



Perform Greedy Best-First Search with the manhattan distance as heuristic for six iterations of the main loop. For each iteration, write down the node that is expanded, the states that are considered for adding to the open list, the content of open list and closed set after the expansion of the state. Make sure to keep the open list ordered and break ties by lower Y value and if that is the same, lower X value. For example, if the following squares have the same heuristic value, they get ordered as $[(3, 0), (2, 1), (4, 1)]$, where $(3, 0)$ is selected first.

You can use the table as below, which already has the first steps.

expanded node	nodes that might get added	open	closed
		[(1,1)]	{}
(1,1)	{(2,1), (1,2)}	[(2,1), (1,2)]	{(1,1)}

Solution:

expanded node	nodes that might get added	open	closed
		[(1,1)]	{}
(1,1)	{(2,1), (1,2)}	[(2,1), (1,2)]	{(1,1)}
(2,1)	{(3,1), (2,0), (1,1)}	[(3,1), (1,2), (2,0)]	{(1,1), (2,1)}
(3,1)	{(3,2), (4,1), (2,1)}	[(4,1), (3,2),] (1,2), (2,0)	{(1,1), (2,1), (3,1)}
(4,1)	{(3,1)}	[(3,2), (1,2), (2,0)]	{(1,1), (2,1), (3,1), (4,1)}
(3,2)	{(3,1)}	[(1,2), (2,0)]	{(1,1), (2,1), (3,1), , (4,1), (3,2)}
(1,2)	{(1,3), (1,1)}	[(1,3), (2,0)]	{(1,1), (2,1), (3,1), (4,1) ,(3,2), (1,2)}

2.2. Consider the **Meeting Point** problem again with the encoding described in Exercise 1.2 above, but this time with the city map below in Figure 1 (the initial state is still $(a = A, e = E, f = F)$).

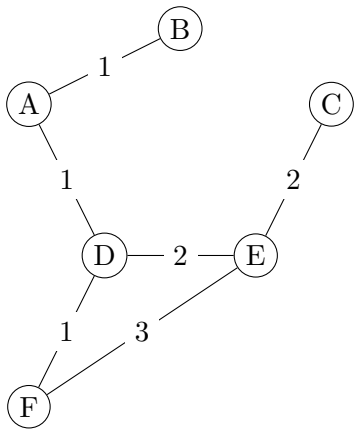


Figure 1: City map with distances between locations.

Let us define heuristic h as the sum of cheapest paths between all pairs friends.

Formally, let $C(x, y)$ be the cheapest path between locations x and y, and let $h(a, e, f)$ denote the heuristic value for the state where Alice is at location a, Emil at location e and Frank at location f. We define h as $h(a, e, f) = C(a, e) + C(a, f) + C(e, f)$.

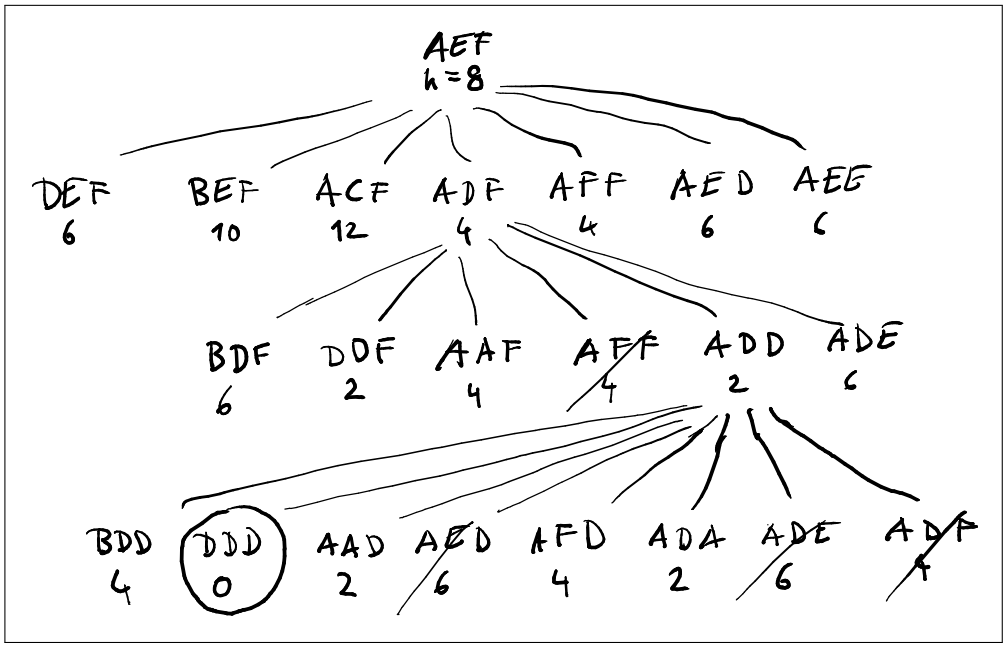
(a) Proof that h is not admissible.

> **Solution:** It suffices to find an example state where h overestimates the optimal solution cost. For example, $(a = D, e = F, f = E)$.

(b) Run Greedy Best-First Search with the heuristic h until termination. When breaking ties, take the lexicographically smallest state (i.e., $(a = A, e = A, f = F)$ before $(a = A, e = B, f = F)$ and so on).

Draw the search tree, mark each state with its h-value, cross states not added to the open list due to duplicate elimination, and circle the goal state in which the search terminates.

> **Solution:** h-value under states, and state encoding more compressed, e.g., $(a = A, e = E, f = F)$ becomes AEF.


AEF
h=8
DEF
6
BEF
10
ACF
12
ADF
4
AFF
4
AED
6
AEG
6
BDF
6
DDF
2
AAF
4
AFF
4
ADD
2
ADE
6
BDD
4
DDD
0
AAD
2
AED
6
AFD
4
ADA
2
ADE
6
ADF
4

2.3. Consider the state space from Figure 2.

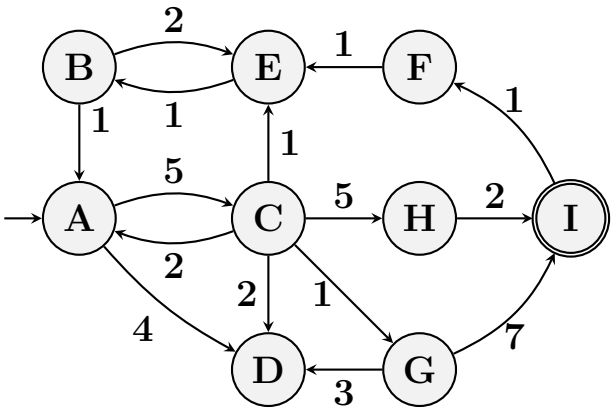


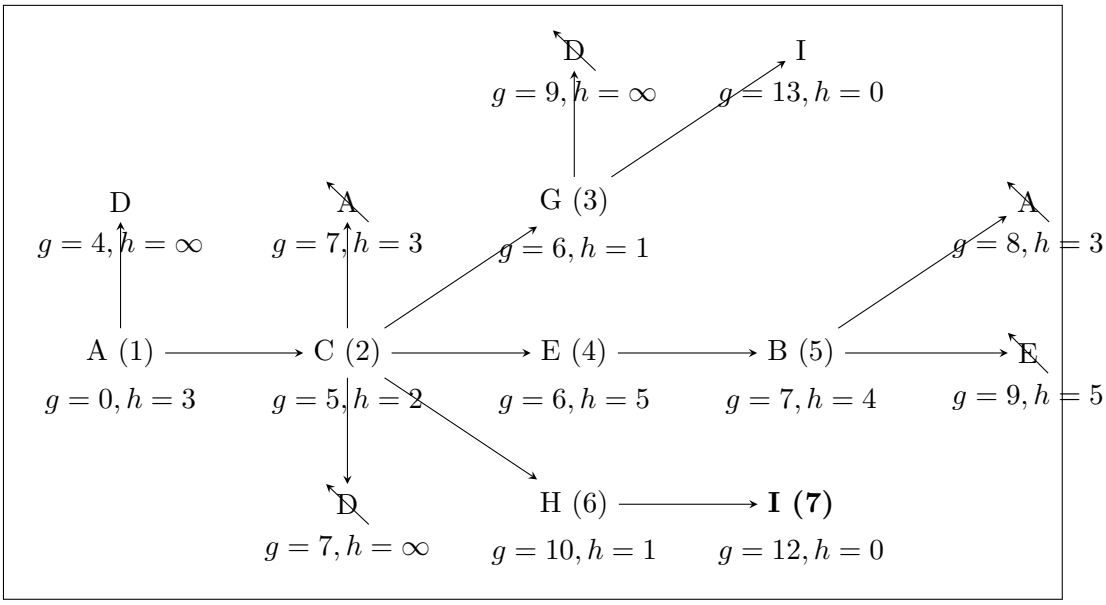
Figure 2: State space used in Exercise 3.

Run A* search on this problem. As a heuristic estimate for a state s, use the minimal number of edges that are needed to reach a goal state from s (or ∞ if s is not solvable, e.g., h(C)=2). When breaking ties, take the lexicographically smallest state (i.e., A before B and so on).

(a) Draw the search tree, mark each state with its h- and g-value, cross states not added to the open list due to duplicate elimination or whose g-value was updated, and mark the goal state in which the search terminates.

(b) Explain why the found solution is optimal.

Solution: The solution is A, C, H, I. A* is guaranteed to return an optimal solution if the heuristic that is being used is admissible. The heuristic is admissible because we do not have any zero-cost edges and thus the length of the shortest path is always lower-bound on the cost of that path.


D
$g = 9, h = \infty$
I
$g = 13, h = 0$
D
$g = 4, h = \infty$
A
$g = 7, h = 3$
G (3)
$g = 6, h = 1$
A
$g = 8, h = 3$
A (1)
$g = 0, h = 3$
C (2)
$g = 5, h = 2$
E (4)
$g = 6, h = 5$
B (5)
$g = 7, h = 4$
E
$g = 9, h = 5$
D
$g = 7, h = \infty$
H (6)
$g = 10, h = 1$
I (7)
$g = 12, h = 0$

2.4. Consider the **Meeting Point** problem as described in Exercise 2.2 above, but on the map below in Figure 3.

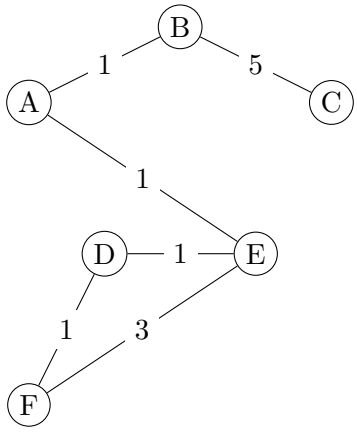


Figure 3: City map with distances between locations.

This time, the heuristic h is computed as the cost of the cheapest path between two locations occupied by at least one friend. For example, $h(a = A, e = E, f = F) = 1$, $h(a = A, e = D, f = D) = 2$, and $h(a = F, e = E, f = F) = 2$.

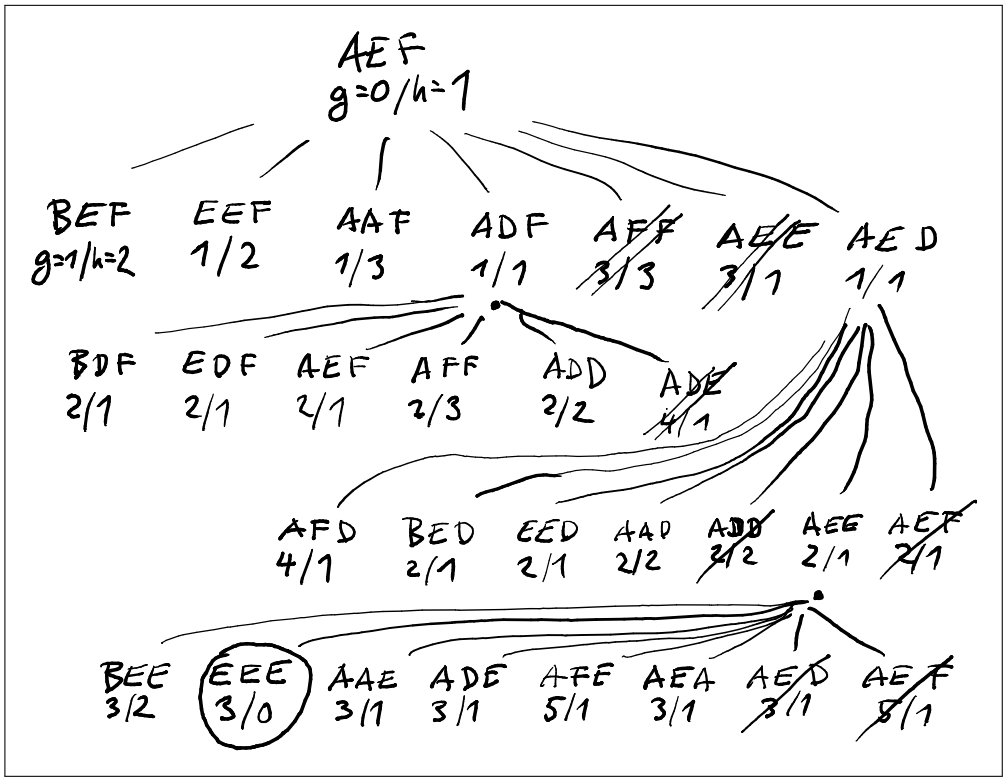
(a) Is the heuristic h consistent? Justify your answer.

> **Solution:** No.
> Consider state $s = (a = D, e = D, f = C)$ and one of its successors $s' = (a = F, e = D, f = C)$. We have that $h(s) = 8$, $h(s') = 1$ and cost of the transition from s to s' is 1. Therefore we have that $h(s) - h(s') = 7$, but for a consistent h we would need to have $h(s) - h(s') \leq 1$.

(b) Run A* search with the aforementioned heuristic on this problem and draw the search tree in the same way as in the previous exercise.

When breaking ties, first prefer states with lower h-value, then lexicographically smaller (i.e., $(a = A, e = B, f = C)$ before $(a = A, e = A, f = C)$ and so on).

> **Solution:**



(c) Is the solution optimal? Justify your answer.

Solution: Yes, because the heuristic is admissible even though it is not consistent.

It is admissible, because for two friends to meet, they need to travel (one or the other or both together) at least the cost of the cheapest path between them.

(d) Can you think of a better admissible heuristic? Describe it and argue why it is admissible and/or consistent.

Solution: For example, we can take the maximum over cheapest paths between all pairs of friends. The argument for admissibility above still holds.

For consistency: Let $s \xrightarrow{a} s'$ be an arbitrary transition in our state space. Let c_{ae}, c_{af}, c_{ef} denote the cost of the cheapest paths in state s between friends Alice-Emil, Alice-Frank, Emil-Frank, respectively. Let $c'_{ae}, c'_{af}, c'_{ef}$ denote the same for the state s'. Therefore, we have that $h(s) = \max\{c_{ae}, c_{af}, c_{ef}\}$ and $h(s') = \max\{c'_{ae}, c'_{af}, c'_{ef}\}$. Since each action moves only one friend, we know that s and s' differ only in one location. Let us, without loss of gen-

erality, assume that Alice moved. Since all costs are positive, we therefore have that $c_{ae} - c'_{ae} \leq \text{cost}(a)$ and $c_{af} - c'_{af} \leq \text{cost}(a)$, and therefore also that $\max\{c_{ae}, c_{af}, c_{ef}\} - \max\{c'_{ae}, c'_{af}, c'_{ef}\} \leq \text{cost}(a)$.

3 Local Search

3.1. Consider the state space from Figure 2 again.

(a) Run the Hill Climbing algorithm on this problem. Use the heuristic function from Exercise 2.3. For each state, provide all applicable actions and the states reachable using these actions. Annotate states with their heuristic value. Specify which node is expanded in each iteration of the algorithm. If the choice of the next state to be expanded is not unique, expand the lexicographically smallest state first. Does the algorithm find a solution? If yes, what is it and is it optimal?

Solution:
Hill climbing returns the solution A, C, G, I, which is not optimal. The expansion order is as follows:

- State A
  $A \to C$: $h(C) = 2 \Leftarrow$ expand
  $A \to D$: $h(D) = \infty$
- State C
  $C \to A$: $h(A) = 3$
  $C \to D$: $h(D) = \infty$
  $C \to E$: $h(E) = 5$
  $C \to G$: $h(G) = 1 \Leftarrow$ expand
  $C \to H$: $h(H) = 1$
- State G
  $G \to D$: $h(D) = \infty$
  $G \to I$: $h(I) = 0 \Leftarrow$ expand
- State I
  $I \to F$: $h(F) = 6$
- I is a local minimum, search stops.

(b) Could hill-climbing stop in a local minimum without finding a solution? If yes, give an example heuristic $h : \{A, B, ..., I\} \to \mathbb{N}_0^+ \cup \{\infty\}$ for the state space depicted in Figure 2 that leads hill-climbing into a local minimum, and explain what happens. If no, explain why.

Solution: Yes it is possible that hill-climbing stops without finding a solution. Consider the following heuristic:

State s	A	B	C	D	E	F	G	H	I
h(s)	2	2	2	2	2	2	2	2	0

Hill climbing starts at A (with $h(A) = 2$) and chooses randomly one of its child nodes (C or D). Since either one of them has an equal heuristic value of 2, Hill Climbing stays at node A, which is then returned as a local minimum without finding the solution.